

Grado en Ciencia e Ingeniería de Datos

Bases de Datos: integración y arquitecturas 2025-26

**Práctica de laboratorio 1: Generación de datos sintéticos
y almacenamiento de información**

Entrega: semana 6 de octubre de 2025, el día antes de la clase de prácticas (ver Moodle)

Modelo de trabajo: la práctica se abordará y entregará por parejas

Objetivos de aprendizaje:

- Aprender a generar datos sintéticos para realizar una ingesta de datos coherente con las necesidades reales del sistema a medir.
- Aprender a almacenar la información en diversos tipos de ficheros y Sistemas Gestores de Bases de Datos (SGBD).
- Aprender a automatizar el proceso de generación de datos sintéticos de forma que el código pueda reutilizarse con diferentes cantidades y formatos de datos.
- Aprender a manejarse de forma autónoma en la lectura y comprensión de la documentación técnica de sistemas gestores de bases de datos, librerías y módulos de programación para el diseño e implementación de software que las emplee.

Descripción:

En el marco de esta práctica, los estudiantes deberán desarrollar e implementar un código que permita la generación de datos sintéticos, así como su almacenamiento en diferentes formatos de ficheros y Sistemas Gestores de Bases de Datos (SGBD).

Al finalizar la actividad, se deberá realizar la entrega del código desarrollado junto con un informe técnico que detalle las decisiones tomadas durante el proceso de implementación y las conclusiones obtenidas.

Estas prácticas permitirán a los estudiantes abordar temas clave en bases de datos, como la generación de datos y la relación entre los mismos.

En esta práctica se deberán generar conjuntos de datos relacionales y almacenar la información en diversos tipos de ficheros y bases de datos.

En particular, los estudiantes deben implementar el código que permita:

1. Generar dos conjuntos de datos de tamaño parametrizable empleando el paquete Faker¹, donde uno de ellos tenga datos relacionados con el otro:
 1. el primero contendrá datos de usuarios (nombre, dni, email, número de teléfono móvil, número de teléfono fijo, dirección, ciudad, código postal² y provincia³).
 2. el segundo contendrá datos de vehículos (matrícula, número de bastidor, año, fabricante, modelo, categoría – urbanos, sedán, berlina, cupé, descapotable, deportivo, todoterreno, monovolumen SUV–) que hayan pertenecido o pertenezcan a usuarios, empleando el DNI como identificador del usuario.
 1. un usuario puede estar relacionado con más de un vehículo.
 2. es posible que deba implementar su propio código para generar parte de la información.
 3. puede resultar útil revisar la documentación de la función `random_element`⁴.
2. Almacenar la información generada en ficheros:
 1. CSV y Parquet⁵: para ambos formatos, se generará un fichero por cada conjunto de datos (usuarios y vehículos) conectando ambos conjuntos de datos por medio de referencias (el campo de la variable de un conjunto de datos será el identificador del otro conjunto de datos).
 2. JSON⁶ y Avro⁷: se deberá abordar la tarea de dos modos diferentes, a saber:
 - (a) un único fichero para todo el conjunto de datos anidando los datos relacionados de forma jerárquica.
 - (b) un fichero por cada conjunto de datos (usuarios y vehículos) conectando ambos conjuntos de datos por medio de referencias (el campo de la variable de un conjunto de datos será el identificador del otro conjunto de datos).
 3. Señala las características más destacadas de cada formato de fichero. ¿Usarías cualquier formato de forma indistinta en cualquier situación? ¿En qué situaciones resultan más aconsejables cada uno de ellos? ¿Qué diferencias encuentras entre usar formatos de almacenamiento basados en filas frente a los basados en columnas? ¿Qué ventajas e inconvenientes encuentras entre almacenar la información mediante ficheros separados o en estructura jerárquica?
3. Almacenar la información generada en SGBD:
 1. SQLite3⁸: sistema de gestión de bases de datos relacional autónomo, sin servidor ni necesidad de configuración, que lee y escribe directamente en archivos de disco.
 2. PostgreSQL⁹: sistema de gestión de bases de datos relacional. Al contrario que el anterior, requiere de un proceso servidor independiente al que las aplicaciones cliente deberán conectarse.

¹ <https://faker.readthedocs.io/en/master/index.html>

² https://raw.githubusercontent.com/inigoflores/ds-codigos-postales-ine-es/master/data/codigos_postales_municipios.csv

³ https://es.wikipedia.org/wiki/Anexo:Provincias_de_Espa%C3%B1a_por_c%C3%B3digo_postal

⁴ https://faker.readthedocs.io/en/master/providers/baseprovider.html#faker.providers.BaseProvider.random_elements

⁵ <https://parquet.apache.org/>

⁶ <https://www.json.org/json-es.html>

⁷ <https://avro.apache.org/>

⁸ <https://docs.python.org/3/library/sqlite3.html>

⁹ <https://pypi.org/project/psycopg2/>

3. MongoDB¹⁰: sistema de gestión de bases de datos de documentos, que emplea colecciones en lugar de guardar los datos en tablas. Al igual que el anterior, requiere de un proceso servidor independiente al que las aplicaciones cliente deberán conectarse.
4. Señala las características más destacadas de cada SGBD. ¿Usarías cualquiera de ellos de forma indistinta en cualquier situación? ¿Para qué situaciones usarías cada uno de ellos?

Debe evitarse el uso de Pandas en todos los apartados, indicando qué implicaciones y/o limitaciones tiene su uso en las diferentes tareas. Como resultado, se debe presentar el código implementado tras haber sido probado con tamaños de datos para 1.000, 10.000 y 100.000 registros, así como un informe con la explicación de lo realizado, indicando la diferencia entre los diferentes modos de almacenamiento de datos y cómo se pueden utilizar en diferentes escenarios.

Entregable

El entregable de las tareas de laboratorio consiste en un fichero comprimido (zip o tgz) con la siguiente nomenclatura BDIA2425-lab1-Nombre1Apellido1_Nombre2Apellido2 (indicando en nombre y apellido de cada integrante). El fichero deberá contener:

- Subdirectorio src que contenga el código desarrollado. El código debe estar bien comentado para facilitar su comprensión, así como estar estructurado siguiendo buenas prácticas de programación e Ingeniería del Software, seleccionando las estructuras de datos apropiadas y los patrones de diseño que considere oportunos. La calidad y claridad del código es parte de la evaluación.
- Un fichero pdf conteniendo un informe en el que se enumeren las tareas realizadas, se discutan las principales cuestiones de interés y (si procede) se informen de los resultados.

¹⁰ <https://www.mongodb.com/es>